AWS Cloud Practitioner
Module 4

Amazon Relational Database Service (RDS)

- Amazon Relational Database Service (RDS) is a managed database service that makes it easy to set up, operate, and scale relational databases in the AWS cloud.

- It provides automated administrative tasks such as provisioning, patching, backups, and scaling.

Key features of (RDS)

1. Multi-Engine Support

Database Engine	Description
Amazon Aurora	MySQL & PostgreSQL-compatible, highly available, and up to 5X faster than MySQL & 3X faster than PostgreSQL.
MySQL	Open-source database, widely used for web applications.
PostgreSQL	Advanced open-source database with support for JSON, GIS, and complex queries.
MariaDB	Open-source, fork of MySQL, known for performance and reliability.
SQL Server	Microsoft's relational database, commonly used in enterprise environments.
Oracle	Commercial database offering high availability and advanced security.

Key features of (RDS)

1.2. Automated Database Administration

- **Automated Backups**: RDS can automatically back up databases based on user-defined retention periods (1–35 days).
- **Snapshots**: Manual backups that can be stored indefinitely.
- **Point-in-Time Recovery (PITR)**: Restore a database to any second within a retention period.
- **Automated Software Patching**: RDS applies patches and updates during the maintenance window without manual intervention.

Key features of (RDS)

1.3. Scalability

- **Vertical Scaling**: Change the instance type to a larger or smaller instance (e.g., from db.t3.micro to db.m5.large).

- **Horizontal Scaling**: Add read replicas for distributing read traffic.

- **Storage Auto-Scaling**: Storage automatically grows up to a specified limit without downtime.

Key features of (RDS)

1.4. High Availability & Disaster Recovery

1. **Multi-AZ Deployments** (Automatic Failover)
    1. Maintains a **synchronous standby replica** in another AZ.
    2. Failover happens automatically if the primary instance fails.
2. **Read Replicas** (Scalability & Performance)
    1. Asynchronous replication of the primary database.
    2. Used to **distribute read traffic** and reduce the load on the primary database.
    3. Available for **Aurora, MySQL, PostgreSQL, and MariaDB**.
3. **Aurora Global Database**
    1. Multi-region database replication for disaster recovery and low-latency global access

Key features of (RDS)

1.5. Security & Compliance

- **IAM Integration**:
  - Manages user access via AWS Identity and Access Management (IAM).
- **Encryption**:
  - **At rest**: Uses **AWS KMS** to encrypt database storage and snapshots.
  - **In transit**: Uses **SSL/TLS** for secure connections.
- **Network Security**:
  - **VPC (Amazon Virtual Private Cloud)** support for database isolation.
  - **Security Groups & Network ACLs** control access to the database.
- **Audit Logging**:
  - Tracks database activity using **AWS CloudTrail** and **RDS logs**.
- **Compliance**:
  - RDS meets industry standards such as PCI-DSS, HIPAA, and FedRAMP.

Key features of (RDS)

1.6. Cost-Effective Pricing

- **On-Demand Instances**:
  - Pay for database usage on an hourly basis.
- **Reserved Instances**:
  - Up to 75% savings for 1-year or 3-year commitments.
- **Aurora Serverless**:
  - Automatically scales based on workload, reducing costs for intermittent usage.

Amazon RDS Deployment Options

Amazon RDS provides different deployment strategies based on business needs.

2.1. Single-AZ Deployment

- Runs a single database instance in one Availability Zone (AZ).
- Suitable for development, testing, or low-priority applications.

2.2. Multi-AZ Deployment (High Availability)

- Creates a standby replica in another Availability Zone.
- Automatic failover in case of instance failure.
- Ideal for production environments.

Amazon RDS Deployment Options

2.3. Read Replicas (Scalability)

- Can be deployed across multiple regions.
- Used for **distributing read traffic** and **offloading queries** from the primary database.
- Can be **promoted** to a standalone database if needed.

Amazon RDS Use cases

3.1. Web & Mobile Applications

- Powering dynamic content, user authentication, and transactional data.

3.2. E-commerce Platforms

- Handling product catalogs, shopping carts, and user orders.

3.3. Enterprise Applications (ERP & CRM)

- Storing and managing business-critical data securely.

3.4. Data Warehousing & Analytics

- Using **Aurora or PostgreSQL** for analytical workloads with parallel queries.

3.5. Multi-Tenant SaaS Applications

- Isolating tenant data with **separate databases or schemas**.

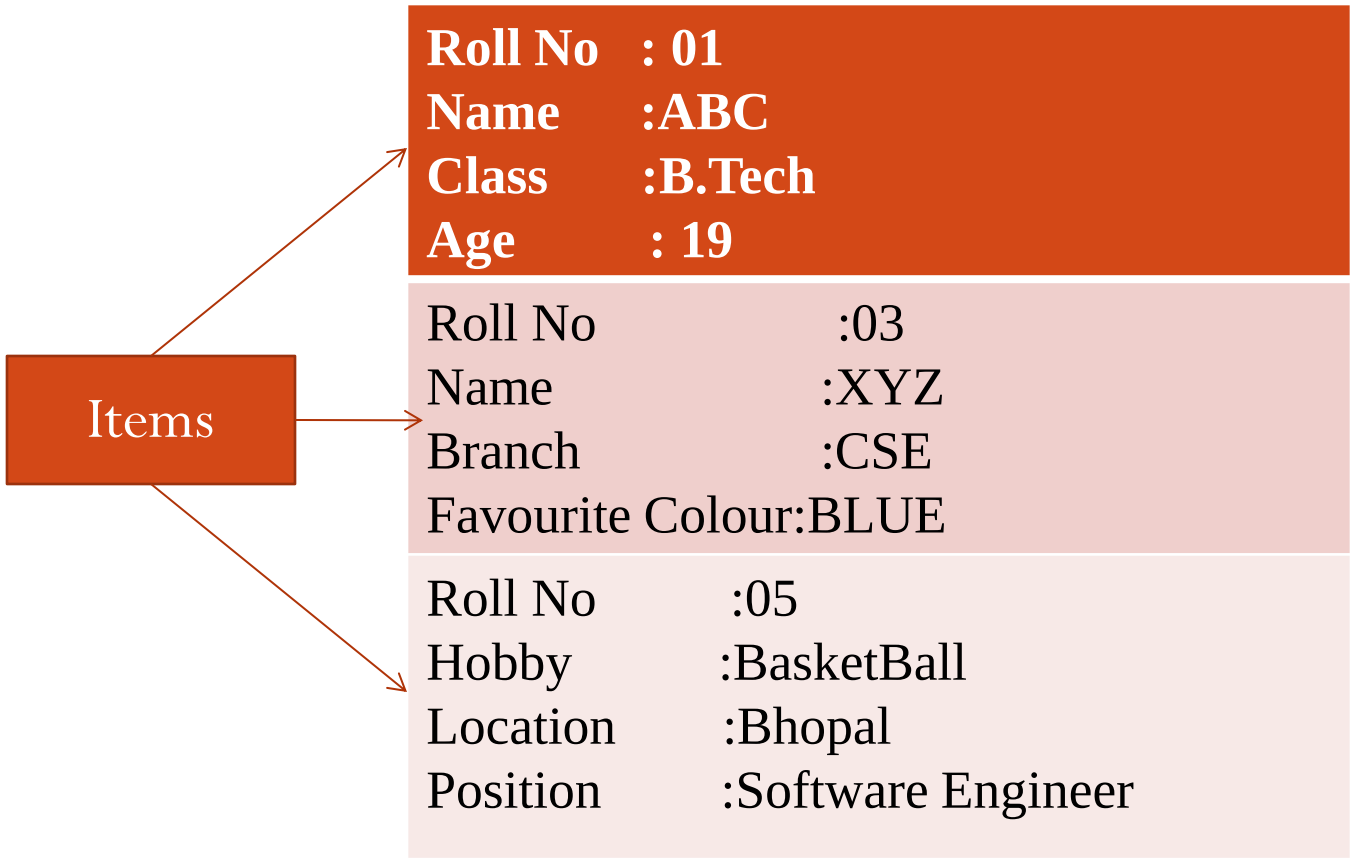
DynamoDB

- Data base Type
  - Unstructured data
  - Semi-Structured data
  - Structured data

- It is a fully managed, key-value NoSQL database designed **to run high-performance applications at any scale**

- DynamoDB offers built-in security, continuous backups, automated multi-region replication, in-memory caching, and data export tools.

DynamoDB: Table



DynamoDB: Table

- Table
  - A Table is a collection of data items
  - Dynamo DB stores data in Tables
- Item
  - Each Table contains multiple items
  - An item is a group of attributes that is uniquely identifiable among all of the other items
  - An item consists of a primary or composit key.
  - Items in DynamoDB are similar into Rows, Records in other DB
- Attribute
  - Each item is composed of one or more attributes.
  - An attribute consists of the attribute name and value or a set of value
  - An attribute is a fundamental data element
  - Note: Aggregate size of an item cannot exceed 400KB including Key and all Attributes

DynamoDB: Table

- Dynamo DB allows low latency read/write access to items ranging from 1 byte to 400 KB.
- Dynamo DB can be used to store pointers to S3 stored objects
- Dynamo DB stores data indexed by a primary key
- Each item has a unique identifier or primary key

DynamoDB: Read Capacity Unit

- In dynamo DB, Read capacity unit (RCU) calculation differs slightly depending on whether you are using strongly consistent or eventually consistent.

Consistent type	RCU Consumption
Strongly consistent	**1 RCU= 1 READ per second for an item upto 4 KB in size**
Eventually Consistent Read	**1 RCU= 2 READs per second for an item upto 4 KB in size**

DynamoDB: Write Capacity Unit

- **1 WCU= 1 WRITE per second for an item upto 1 KB in size**

- **If the item is larger, the WCU consumption is rounded upto next KB multiple.**

DynamoDB: Pricing

- Reads are cheaper than Write
- You pay for-
  - Each table's provisioned read/write throughput (hourly rates)
  - You are charged for provisioned throughput regardless whether you use it or not
  - Indexed data storage
  - Internet data transfer

DynamoDB can do 10,000 write capacity unit/sec or 10,000 read capacity unit per second per table.

Amazon Redshift

- Amazon **Redshift** is a fully managed **cloud data warehouse** service designed for **analytical workloads**.

- It is optimized for **OLAP (Online Analytical Processing)** and is used for running complex queries on **large datasets**.

Key features of Amazon Redshift

•**Columnar Storage**

•Stores data in **columns** instead of rows, improving query performance for analytical workloads.

•**Massively Parallel Processing (MPP)**

•Distributes query execution across multiple nodes, enabling fast data retrieval.

Key features of Amazon Redshift

- **High Performance with Query Optimization**
- Uses advanced query planning, caching, and compression to speed up execution.

- **Scalability & Elasticity**
- Scales horizontally by adding more nodes or vertically by upgrading instance types.
- Redshift Serverless allows on-demand scaling without manual cluster management.

- **Data Integration**
- Supports data ingestion from **S3, DynamoDB, RDS, Kinesis, and on-premises databases**.
- Can query data from **Amazon S3 (via Redshift Spectrum)** without loading it into Redshift.

Key features of Amazon Redshift

- **Cost Optimization**
- **Pay-as-you-go** pricing for storage and compute.
- Uses **compressed storage** to reduce costs.
- Serverless option eliminates the need for provisioning clusters.

- **Security & Compliance**
- **Encryption (AWS KMS, SSL/TLS)** for data protection.
- **IAM integration** for access control.
- **VPC support** for network isolation.

Amazon Redshift, Amazon RDS, Dynamo DB

Feature	Amazon Redshift	Amazon RDS	DynamoDB
Type	Data Warehouse (OLAP)	Relational Database (OLTP)	NoSQL (Key-Value & Document Store)
Best for	Analytics, BI, reporting	Transactional apps, structured data	Fast lookups, unstructured data
Schema	Schema-based (relational)	Schema-based (relational)	Schema-less (NoSQL)
Query Language	SQL (PostgreSQL-based)	SQL (MySQL, PostgreSQL, etc.)	Key-value or document-based queries
Scalability	MPP architecture (horizontal scaling)	Vertical scaling, Read Replicas	Auto-scaling (serverless)
Performance	Optimized for complex queries	Optimized for transactions	Optimized for key-value lookups

AWS Development Tools

Introduction

- Overview of AWS Development Tools
- Importance of automation in cloud development
- How AWS tools enhance DevOps practices

AWS Development Tools

What is AWS CloudFormation?

- **Definition**: AWS CloudFormation is an Infrastructure as Code (IaC) service that helps automate the provisioning of AWS resources.
- **Purpose**: It allows developers to define and deploy AWS infrastructure using templates.
- **Infrastructure as Code (IaC) Concept**: Helps manage infrastructure in a structured, repeatable way using JSON or YAML templates.

AWS Development Tools

Benefits of AWS CloudFormation

- **Automated provisioning**: Eliminates manual infrastructure setup.
- **Version control for infrastructure**: Enables tracking changes and rollback.
- **Reusability of templates**: Use predefined templates across multiple deployments.
- **Cost optimization**: Automates resource management, reducing unnecessary expenses.

AWS Development Tools

AWS CloudFormation Key Components

- **Templates**: JSON or YAML files defining resources and configurations.
- **Stacks**: A collection of AWS resources managed as a single unit.
- **StackSets**: Allow managing multiple stacks across different AWS accounts and regions.
- **Change Sets**: Preview changes before applying updates.

AWS Development Tools

AWS CloudFormation Use Cases

- **Automating infrastructure deployment**: Reduces human errors and accelerates provisioning.
- **Managing multi-region deployments**: Ensures consistency across different locations.
- **Disaster recovery**: Enables quick restoration of resources.
- **Security compliance**: Implements predefined security standards.

AWS Development Tools

What is AWS Elastic Beanstalk?

- **Definition**: AWS Elastic Beanstalk is a Platform as a Service (PaaS) that simplifies application deployment.
- **Purpose**: Automatically handles deployment, scaling, monitoring, and load balancing.
- **Supported languages**: Java, .NET, PHP, Node.js, Python, Ruby, Go, and Docker.

AWS Development Tools

Benefits of AWS Elastic Beanstalk

- **Simplified deployment process**: No need to manage infrastructure manually.
- **Built-in scaling and monitoring**: Automatically adjusts resources based on traffic.
- **Supports multiple languages & frameworks**: Compatible with various development stacks.
- **Integration with AWS services**: Works seamlessly with RDS, S3, and IAM.

AWS Development Tools

AWS Elastic Beanstalk Workflow

1. **Application Deployment**: Upload application code to Beanstalk.

2. **Configuration & Scaling**: Automatically provisions EC2 instances and scales based on demand.

3. **Monitoring & Logs**: Uses Amazon CloudWatch and health dashboards for tracking performance.

4. **Updates & Rollbacks**: Supports version control and easy rollbacks.

- .

AWS Development Tools

AWS Elastic Beanstalk Use Cases

- **Web application hosting**: Deploy web apps with minimal effort.
- **Microservices deployment**: Supports containerized applications using Docker.
- **Development and testing**: Quickly deploy test environments.
- **E-commerce platforms**: Handles traffic spikes efficiently.
- .

AWS Development Tools

Introduction to CI/CD Pipelines

- **Continuous Integration (CI)**: Automates code integration and testing.
- **Continuous Deployment (CD)**: Ensures smooth and automated deployment to production.
- **Importance**: Reduces manual effort, increases development speed, and improves reliability.

AWS Development Tools

Introduction to CI/CD Pipelines

- **Continuous Integration (CI)**: Automates code integration and testing.
- **Continuous Deployment (CD)**: Ensures smooth and automated deployment to production.
- **Importance**: Reduces manual effort, increases development speed, and improves reliability.

AWS CI/CD Tools Overview

- **AWS CodeCommit**: Managed Git repository service.
- **AWS CodeBuild**: Fully managed build service.
- **AWS CodeDeploy**: Automates application deployment.
- **AWS CodePipeline**: Orchestrates CI/CD workflows.

AWS Development Tools

What is AWS CodePipeline?

- **Definition**: AWS CodePipeline is a CI/CD automation service that enables continuous integration and delivery.
- **Purpose**: Automates software release processes from source code to deployment.
- **Role in CI/CD workflows**: Orchestrates source, build, test, and deploy stages.

Benefits of AWS CodePipeline

- **Fully managed CI/CD automation**: Eliminates manual intervention.
- **Integration with AWS & third-party tools**: Works with GitHub, Jenkins, and Bitbucket.
- **Parallel execution of deployments**: Speeds up releases and reduces errors.
- **Automated testing and rollback**: Ensures reliable deployments.

AWS Development Tools

AWS CodePipeline Workflow

1. **Source Stage**: Fetches code from CodeCommit, GitHub, or S3.
2. **Build Stage**: Uses CodeBuild to compile and test code.
3. **Test Stage**: Runs automated tests to detect errors.
4. **Deploy Stage**: Deploys the final build to Elastic Beanstalk, EC2, or other AWS services.

AWS CodePipeline Use Cases

- **Automating application releases**: Ensures fast and consistent deployments.
- **Enabling DevOps practices**: Improves collaboration and efficiency.
- **Continuous security checks**: Integrates security scanning into pipelines.
- **Multi-environment deployment**: Supports staging, QA, and production workflows.

Comparison of AWS Cloudformation, Beanstalk & CodePipeline

Feature	CloudFormation	Beanstalk	CodePipeline
Purpose	Infrastructure as Code	Application Deployment	CI/CD Automation
Type	IaC Tool	PaaS	DevOps Tool
Automation Level	High	Medium	High
Use Case	Provisioning Resources	Deploying Applications	Automating Releases
Customization	High	Limited	Medium

AWS Development Tools

Best Practices for Using AWS Development Tools

- **Define clear IaC templates**: Standardize infrastructure deployments.
- **Automate testing in CI/CD**: Prevent bugs from reaching production.
- **Monitor application performance**: Use AWS CloudWatch and X-Ray.
- **Enable security best practices**: Use IAM roles, encryption, and logging.

Challenges & Solutions

- **Managing large CloudFormation stacks** → Use modular templates with nested stacks.
- **Debugging Beanstalk applications** → Leverage logs and monitoring tools.
- **Handling deployment failures in CodePipeline** → Implement rollback strategies.

AWS Development Tools

- **Summary**: AWS provides a robust set of development tools for automating infrastructure, application deployment, and CI/CD pipelines.
- **Importance**: Improves efficiency, reduces errors, and accelerates development cycles.
- **Final Thought**: Choosing the right tool depends on specific use cases and requirements.